

Interface কী?

Interface হলো হার্ডওয়্যার এবং সফটওয়্যারের সমন্বয়, যা CPU এবং peripheral device (যেমন কীবোর্ড, প্রিন্টার, মাউস) এর মধ্যে যোগাযোগ করায়।

কারণ CPU আর peripheral গুলোর কাজ করার ধরন এক নয়, তাই তাদের মধ্যে সরাসরি যোগাযোগ সম্ভব না। এই সমস্যাটা সমাধান করে Interface।

Interface এর দুইটা অংশ

1. Hardware অংশ

যেটায় আমরা কেবল লাগাই

উদাহরণ: USB port, HDMI port

2. Software অংশ

যেটা ডিভাইস চালানোর জন্য ইনস্টল করতে হয়

উদাহরণ: Printer driver, Scanner driver

Peripheral ডিভাইসগুলো কেন Interface ছাড়া কাজ করতে পারে না?

Peripheral ডিভাইসগুলোর বৈশিষ্ট্য একেক রকম। যেমন:

- ডেটা পাঠানোর গতি (data transfer rate) ভিন্ন
- আলাদা আলাদা code ও control signal ব্যবহার করে
- কেউ serial data পাঠায়, কেউ parallel data
- অনেক ডিভাইস CPU এর চেয়ে বেশি voltage এ কাজ করে
- সব peripheral CPU এর তুলনায় অনেক ধীরগতির

👉 তাই CPU আর peripheral এর মাঝে **Interface দরকার**, যাতে এই পার্থক্যগুলো সামলানো যায়।

Interface এর প্রধান কাজগুলো (Exam এ খুব গুরুত্বপূর্ণ)

1. Buffering

Buffer হলো Interface এর ভেতরে থাকা এক ধরনের RAM।

এর কাজ:

- CPU আর peripheral এর মাঝখানে ডেটা সাময়িকভাবে জমা রাখা
- CPU অনেক দ্রুত, peripheral ধীর
তাই buffer ডেটা জমা করে রাখে, পরে CPU সেটাকে নিজের গতিতে প্রসেস করে

👉 এর ফলে CPU কে peripheral এর জন্য অপেক্ষা করতে হয় না।

এক লাইনের এক্সাম উত্তর:

Buffering CPU এবং peripheral এর গতির পার্থক্য সমন্বয় করে।

2. Serial ও Parallel ডেটা রূপান্তর

Data Transmission কী?

এক ডিভাইস থেকে আরেক ডিভাইসে ডেটা পাঠানোই data transmission।

Serial Data Transmission

- একটার পর একটা এক বিট করে ডেটা পাঠানো হয়
- গতি কম
- কিন্তু দূরত্ব বেশি হলে কার্যকর

উদাহরণ: USB, Bluetooth

Parallel Data Transmission

- একসাথে অনেক বিট (১৬ বা ৩২ বিট) পাঠানো হয়
- খুব দ্রুত
- কিন্তু শুধু অল্প দূরত্বে কাজ করে

উদাহরণ: CPU এর ভিতরের bus system

- 👉 CPU এর ভিতরের সব bus **parallel**
- 👉 কিন্তু অনেক peripheral **serial** ডেটা পাঠায়

তাই:

- Interface প্রথমে serial ডেটা buffer এ রাখে
 - তারপর সেটাকে parallel ডেটায় convert করে
 - তারপর CPU এর কাছে পাঠায়
-

3. Analogue ↔ Digital Conversion

- CPU শুধু Digital data বোঝে
- কিন্তু অনেক ডিভাইস Analogue data ব্যবহার করে

উদাহরণ:

- Microphone → Analogue
- Speaker → Analogue

👉 Interface analogue কে digital বানায়, আবার digital কে analogue বানায়

4. Voltage Conversion

- Peripheral অনেক সময় CPU এর চেয়ে বেশি voltage ব্যবহার করে
 - Interface voltage কম বা বেশি করে CPU-র উপযোগী করে
-

5. Protocol Conversion

- বিভিন্ন ডিভাইস বিভিন্ন নিয়মে (protocol) ডেটা পাঠায়
 - Interface সেই protocol কে CPU-র বোঝার মতো করে পরিবর্তন করে
-

6. Status Signal Handling

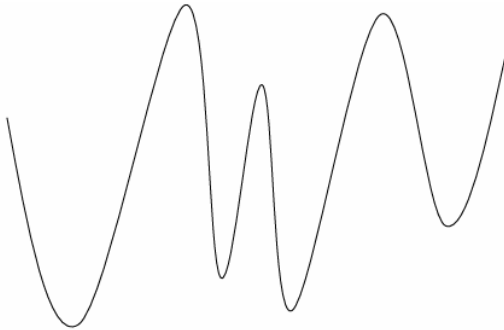
- Peripheral থেকে CPU কে জানায়:
 - ডিভাইস ready কিনা
 - busy কিনা
 - error হয়েছে কিনা

উদাহরণ: Printer busy signal

Analogue ও Digital ডেটা রূপান্তর (Converting data to and from analogue and digital forms)

Analogue Signal কী?

- অনেক electrical signal হলো **analogue signal**
- Analogue signal দুটি সীমার মধ্যে **ক্রমাগত পরিবর্তিত হয়**
- এটি মসৃণ (smooth) তরঙ্গের মতো দেখতে হয়



উদাহরণ:

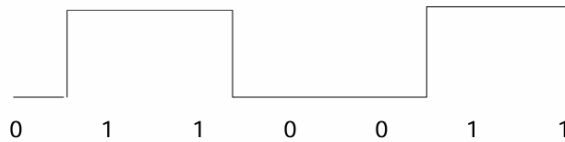
- Microphone এর শব্দ
- Temperature sensor
- Speaker এর সাউন্ড

👉 Peripheral থেকে আসা অনেক signal **analogue**, কিন্তু CPU এগুলো বুঝতে পারে না।

Digital Signal কী?

- Computer শুধু **digital signal** বুঝতে পারে
- Digital signal এর মাত্র **দুটি মান**:
 - ON = 1
 - OFF = 0
- Digital signal মানে অনেকগুলো 1 আর 0 এর সিরিজ

👉 CPU সব কাজ করে 0 আর 1 দিয়ে।



ADC এবং DAC

DAC (Digital to Analogue Converter)

- Computer থেকে বের হওয়া signal হলো **digital**
- কিন্তু অনেক peripheral **analogue signal** বোঝে
- তাই Interface এর মধ্যে থাকা **DAC** digital signal কে analogue signal এ রূপান্তর করে

উদাহরণ:

- Computer → Speaker
- Digital audio → Analogue sound wave

ADC (Analogue to Digital Converter)

- Peripheral থেকে আসা signal অনেক সময় **analogue**
- CPU বুঝতে পারে শুধু **digital**
- তাই **ADC** analogue signal কে digital signal এ রূপান্তর করে

উদাহরণ:

- Microphone → Computer
- Analogue sound → Digital data

Voltage Conversion

- Peripheral device এবং CPU **একই voltage** ব্যবহার করে না
- CPU সাধারণত কম voltage এ কাজ করে

- অনেক peripheral তুলনামূলক বেশি voltage ব্যবহার করে

👉 Interface voltage কে এমনভাবে পরিবর্তন করে যাতে:

- Peripheral CPU কে ক্ষতি না করে
- দুই দিকেই নিরাপদভাবে data আদান-প্রদান হয়

এক লাইনের এক্সাম উত্তর:

Voltage conversion CPU ও peripheral এর ভিন্ন voltage level সমন্বয় করে।

Protocol Conversion

Protocol কী?

- Protocol হলো **কিছু নির্দিষ্ট নিয়ম**, যেগুলো অনুসরণ করে ডেটা পাঠানো হয়
- এতে থাকে:
 - ডেটা পাঠানোর ক্রম
 - control signal
 - error checking নিয়ম

উদাহরণ:

- USB protocol
 - HDMI protocol
-

Protocol Conversion কী?

- Peripheral এবং computer অনেক সময় **ভিন্ন protocol** ব্যবহার করে
- Interface সেই protocol কে **একে অপরের বোঝার মতো করে রূপান্তর করে**

👉 ফলে computer এবং peripheral ঠিকভাবে যোগাযোগ করতে পারে।

Handling of Status Signal

Status Signal কী?

- Peripheral CPU কে জানায় তার বর্তমান অবস্থা
- যেমন:
 - Ready
 - Busy
 - Error

উদাহরণ:

- Printer busy
- Paper missing
- Device not connected

👉 এই তথ্য:

- CPU কে সিদ্ধান্ত নিতে সাহায্য করে
- User কে সমস্যার কথা জানায়

Buffers এবং Spoolers কী?

Buffers এবং Spoolers সাধারণত **printer** এর সাথে বেশি ব্যবহার হয়, তবে এগুলো **শুধু printer-এর জন্য নয়**—যেকোনো ধীরগতির peripheral এর ক্ষেত্রেই ব্যবহার করা যায়।

মূল উদ্দেশ্য একটাই:

👉 CPU আর ধীরগতির peripheral এর মাঝের গতির পার্থক্য সমাধান করা।

Buffer কী?

Buffer হলো মেমরির (RAM) একটি অংশ, যেটা **computer আর peripheral এর মধ্যে ডেটা আদান-প্রদানের সময় সাময়িকভাবে ডেটা ধরে রাখে।**

Buffer এর কাজ

- ডেটা **temporary storage** হিসেবে রাখা
- CPU আর peripheral এর মধ্যে **সংযোগ তৈরি করা**
- দুইটার কাজ করার **গতির পার্থক্য সামলানো**

Buffer কেন ব্যবহার করা হয়?

1. CPU অনেক দ্রুত, Peripheral ধীর

- CPU খুব দ্রুত কাজ করে
- Printer, keyboard, mouse ইত্যাদি অনেক ধীরে কাজ করে

👉 Buffer CPU-র গতি আর peripheral-এর গতি মিলিয়ে দেয়।

2. Printer এর ক্ষেত্রে Buffer

- CPU খুব দ্রুত ডেটা buffer-এ পাঠিয়ে দেয়
- Buffer ধীরে ধীরে printer-এ ডেটা পাঠায়
- Printer নিজের ক্ষমতা অনুযায়ী ডেটা নেয়

👉 এতে CPU বসে থাকে না, নিজের কাজ চালিয়ে যেতে পারে।

3. CPU Interrupt কম হয়

- Keyboard থেকে আসা ডেটা সরাসরি CPU-তে গেলে বারবার interrupt হতো
- Buffer ডেটা জমা করে রাখে
- যখন অনেক ডেটা জমে, তখন CPU প্রসেস করে

👉 এতে CPU-র উপর চাপ কমে।

Spooler কী?

Spooler হলো ডেটা আদান-প্রদানের আরেকটা পদ্ধতি, যেটা সাধারণত খুব ধীরগতির peripheral, বিশেষ করে printer-এর জন্য ব্যবহৃত হয়।

Spooling কীভাবে কাজ করে?

- Printer-এর জন্য পাঠানো ডেটা আগে hard disk বা অন্য storage-এ রাখা হয়

- CPU যখন ফাঁকা (idle) থাকে, তখন ডেটা printer-এ পাঠানো হয়
 - একে বলা হয় **background printing**
-

Spooling এর সুবিধা

- CPU মুক্ত থাকে
- User একসাথে একাধিক কাজ করতে পারে
- System performance উন্নত হয়

👉 CPU printer-এর জন্য অপেক্ষা করে না।

Buffer বনাম Spooler (Exam খুব গুরুত্বপূর্ণ)

Buffer এর সীমাবদ্ধতা

- Buffer থাকে RAM-এ
 - RAM এর জায়গা সীমিত
 - খুব বড় ডেটার জন্য buffer যথেষ্ট নাও হতে পারে
-

Spooler এর সীমাবদ্ধতা

- Spooler কাজ করে যখন CPU idle থাকে
 - CPU যদি খুব ব্যস্ত থাকে, তাহলে printing ধীর হয়ে যায়
-

কোনটা ভালো?

👉 সবচেয়ে ভালো সমাধান হলো Buffer এবং Spooler দুটোই একসাথে ব্যবহার করা।

- Buffer দ্রুত ডেটা আদান-প্রদান করে
 - Spooler বড় ডেটা নিরাপদে ধরে রাখে
-

Interface Standards কী?

Interface standard হলো নির্দিষ্ট কিছু নিয়ম ও প্রযুক্তি, যার মাধ্যমে **computer** এবং **peripheral device** একে অপরের সাথে ঠিকভাবে সংযুক্ত ও যোগাযোগ করতে পারে।

👉 Standard থাকলে আলাদা কোম্পানির device হলেও একসাথে কাজ করতে পারে।

কেন Interface Standards দরকার?

- Computer manufacturer যদি standard ব্যবহার করে, তাহলে সেই computer অনেক ধরনের peripheral-এর সাথে কাজ করতে পারবে
- Peripheral manufacturer যদি standard অনুসরণ করে, তাহলে তার device অনেক computer-এর সাথে চলবে
- ফলে system হয় **compatible**

👉 মূল কথা: **Compatibility** নিশ্চিত করে

গুরুত্বপূর্ণ Interface Standards (Exam ফোকাস)

1. RS232

- **Recommended Standard**
 - **Serial communication**
 - পুরনো device যেমন modem এ ব্যবহৃত হতো
-

2. SCSI

- **Small Computer Systems Interface**
 - **Parallel interface**
 - High-speed device যেমন hard disk, scanner
-

3. IDE

- **Integrated Drive Electronics**
 - Hard disk এবং CD/DVD drive সংযোগে ব্যবহৃত
 - পুরনো standard
-

4. SATA

- **Serial Advanced Technology Attachment**
 - IDE এর উন্নত সংস্করণ
 - গতি: প্রায় 1.5 Gbps বা তার বেশি
 - Modern hard disk ও SSD তে ব্যবহৃত
-

5. IEEE

- **Institute of Electrical and Electronic Engineers**
 - বিভিন্ন standard তৈরি করে
 - উদাহরণ: FireWire (IEEE 1394)
-

6. MIDI

- **Musical Instrument Digital Interface**
 - Musical instrument এবং computer এর মধ্যে data transfer
 - Sound data পাঠায়, actual sound না
-

7. PCI

- **Peripheral Component Interconnect**
 - Motherboard এর ভিতরের expansion card সংযোগ
 - Graphics card, sound card
-

8. PCMCIA

- **Personal Computer Memory Card International Association**
- Laptop এ expansion card ব্যবহৃত

- আগে বেশি জনপ্রিয় ছিল
-

9. USB 1 & 2

- **Universal Serial Bus**
 - Keyboard, mouse, printer, pen drive
 - Plug and play সুবিধা
-

পরীক্ষার জন্য এক লাইনের সারাংশ

Interface standards বিভিন্ন device কে একে অপরের সাথে compatible করে।

Wireless Communication

Wireless Communication কী?

তার ছাড়া (wire ছাড়া)

radio waves ব্যবহার করে ডেটা আদান-প্রদান করাকে wireless communication বলে।

বর্তমান সবচেয়ে জনপ্রিয়:

- **Bluetooth**
 - **WiFi**
-

Bluetooth এবং WiFi এর সাধারণ বৈশিষ্ট্য

- দুটোই **radio waves** ব্যবহার করে
 - দেয়াল ও বেশিরভাগ বস্তু ভেদ করে যেতে পারে
 - Device গুলোকে মুখোমুখি থাকতে হয় না
(Infrared এর মতো না)
-

Bluetooth

Bluetooth কী?

- **Short-range wireless communication**
- Personal device এর মধ্যে সংযোগে ব্যবহৃত

Bluetooth এর ব্যবহার

- Mobile phone ও headset
- Laptop ও mobile
- Wireless keyboard ও mouse

Bluetooth এর বৈশিষ্ট্য

- কম দূরত্ব
 - কম power ব্যবহার
 - ভবিষ্যৎ সংস্করণে:
 - Speed প্রায় 2 Mbps
 - দূরত্ব আরও বেশি
-

WiFi

WiFi কী?

- **Wireless Fidelity Alliance**
- প্রধানত Wireless LAN (WLAN) এ ব্যবহৃত

WiFi এর বৈশিষ্ট্য

- Range: প্রায় 15–50 meter
 - Speed: প্রায় 5–20 Mbps
 - Internet, network sharing
-

Bluetooth বনাম WiFi (Exam-friendly)

বিষয়	Bluetooth	WiFi
Range	কম	বেশি
Speed	কম	বেশি
Power	কম	বেশি
ব্যবহার	Personal device	Network / Internet

পরীক্ষার জন্য চূড়ান্ত সারাংশ

- **Interface standards compatibility** নিশ্চিত করে
- **Bluetooth** short-range personal communication
- **WiFi** wireless networking এর জন্য
- দুটোই **radio waves** ব্যবহার করে

Microprocessor Data Type

Microprocessor Data Type

1 Bit

- সবচেয়ে ছোট তথ্যের একক
- এর মান হয় শুধু:
 - 0
 - 1

👉 কম্পিউটার সব তথ্য bit দিয়ে প্রকাশ করে।

2 Byte

- 8 bit = 1 byte
- সাধারণত একটি character সংরক্ষণে 1 byte লাগে

উদাহরণ:

01000001 = 1 byte

3 Word

- Microprocessor একসাথে যত bit নিয়ে কাজ করে তাকে Word বলে
- এটি processor-এর উপর নির্ভর করে

উদাহরণ:

- 8-bit processor → 8 bit word
- 16-bit processor → 16 bit word
- 32-bit processor → 32 bit word

👉 Word size যত বড়, processor তত বেশি ডেটা একসাথে প্রক্রিয়া করতে পারে।

Unsigned এবং Signed Binary Integer

Unsigned Binary Integer

- এখানে কোনো + বা - চিহ্ন নেই
- শুধু মান (magnitude) থাকে
- Leading bit সংখ্যার অংশ

উদাহরণ:

Memory address সবসময় unsigned হয়।

উদাহরণ ১

Decimal 92 কে unsigned binary তে প্রকাশ:

92 ÷ 2 করে করলে পাওয়া যায়:

1011100₂

এটি ৭-bit binary magnitude।

উদাহরণ ২

5-bit unsigned number এর range কত?

Formula:

Range = 0 থেকে $(2^n - 1)$

এখানে $n = 5$

= 0 থেকে $(2^5 - 1)$

= 0 থেকে 31

Minimum = 00000 (0)

Maximum = 11111 (31)

Signed Binary Integer

এখানে সংখ্যা ধনাত্মক (+) বা ঋণাত্মক (-) হতে পারে।

Signed number প্রকাশের তিনটি পদ্ধতি আছে:

(a) Sign-Magnitude Form

- প্রথম bit = sign bit
 - 0 = positive
 - 1 = negative
- বাকি bit = magnitude

Range formula:

$-(2^{n-1} - 1)$ থেকে $+(2^{n-1} - 1)$

উদাহরণ: 6-bit

Range = -31 থেকে +31

+31 = 0 11111

-31 = 1 11111

! সমস্যা:

Zero এর দুইটা রূপ আছে

+0 = 0 00000

-0 = 1 00000

(b) 1's Complement Form

- Negative number পেতে সব bit উল্টাতে হয় (0→1, 1→0)

Range:

$-(2^{n-1} - 1)$ থেকে $+(2^{n-1} - 1)$

6-bit হলে:

Range = -31 থেকে +31

+31 = 0 11111

-31 = 1 00000

! এখানেও zero এর দুইটা রূপ আছে:

+0 = 0 00000

-0 = 1 11111

(c) 2's Complement Form (সবচেয়ে গুরুত্বপূর্ণ)

Negative বের করার নিয়ম:

- সব bit উল্টাও (1's complement)
- 1 যোগ করো

Range formula:

$-(2^{n-1})$ থেকে $+(2^{n-1} - 1)$

6-bit হলে:

Range = -32 থেকে +31

+31 = 0 11111

-32 = 1 00000

👉 এখানে zero এর একটাই রূপ আছে:

$0 = 0\ 00000$

☑ তাই 2's complement সবচেয়ে বেশি ব্যবহৃত।

BCD (Binary Coded Decimal)

- Decimal এর প্রতিটি digit আলাদা করে 4-bit binary তে প্রকাশ করা হয়

উদাহরণ:

Decimal 59

$5 \rightarrow 0101$

$9 \rightarrow 1001$

তাহলে BCD = 0101 1001

👉 এটা pure binary না, digit-wise conversion।

ASCII

- Character কে binary তে প্রকাশ করার code
- প্রতিটি character এর আলাদা code থাকে
- সাধারণত 7 বা 8 bit ব্যবহার হয়

উদাহরণ:

A = 65 (decimal)

Binary = 01000001

Floating Point Number

- ভগ্নাংশ বা বড় সংখ্যা প্রকাশে ব্যবহার হয়
- Scientific notation এর মতো কাজ করে

উদাহরণ:

3.75

0.375×10^1

Computer এটাকে mantissa এবং exponent আকারে সংরক্ষণ করে।

From Microprocessor from 5th semester

Convert the following decimal numbers into single-precision floating-point numbers:

Single-precision (32-bit) floating-point representation is defined by IEEE 754 standard with 1 sign bit, 8 exponent bits, and 23 mantissa bits.

(a) +1.5

- Single-precision: 0 01111111 10000000000000000000000

১.৫ কে IEEE 754 ফর্মে নিতে কেন Normalize করি?

আমরা floating-point সংখ্যাটাকে সবসময় এই format-এ লিখি:

$$1. (\text{fraction}) \times 2^{\text{exponent}}$$

মানে, বাম দিকে সবসময় ১ থাকবে — এটাকেই বলে normalized form.

✨ এবার ১.৫ কে binary বানাই

- $1_{10} \rightarrow 1$
- $0.5_{10} \rightarrow .1$

So,

$$1.5_{10} = 1.1_2 \quad \checkmark$$

✨ Normalize করি

আমরা এমনভাবে লিখবো যাতে বাম পাশে ঠিক ১ থাকে:

$1.1_2 \rightarrow$ এটা এমনিতেই $1.x$ আকারে আছে

👉 তাই normalized form =

$$1.1 \times 2^0$$

এখানে exponent = 0 (কারণ আমরা দশমিক বিন্দু সরাইনি)

এখানে exponent = 0 (কারণ আমরা দশমিক বিন্দু সরাইনি)

🔥 IEEE-754 এ exponent এর সাথে 127 bias যোগ করতে হয়

Formula:

$$\text{Biased exponent} = \text{Actual exponent} + 127$$

- Actual exponent = 0
- তাই Biased exponent = $0 + 127 = 127$
- 127 এর 8-bit binary = 01111111

📦 এখন সবকিছু মিলিয়ে final format

Sign	Exponent (8-bit)	Mantissa (23-bit fraction)
0	01111111	10000000000000000000000

(কেননা 1.1 → fraction part = .1, তারপর ২৩-bit পর্যন্ত zero দিয়ে পূরণ)

👉 Final IEEE 754 single-precision representation =
0 01111111 10000000000000000000000

(b) -10.625

- Single-precision: 1 10000001 01010100000000000000000

Sure! চল প্রথমে ধাপে ধাপে দেখি কীভাবে -10.625 কে IEEE-754 single-precision (32-bit) floating-point format-এ রূপান্তর করা হয়।

❖ Step-1: Sign determine করি

- সংখ্যা negative, তাই
➤ Sign bit = 1

❖ Step-2: Magnitude (+10.625) কে binary তে রূপান্তর করি

♦ Integer part $\rightarrow 10_{10}$:

Divide by 2	Quotient	Remainder
10	5	0
5	2	1
2	1	0
1	0	1

👉 Bottom $\rightarrow$ top $\rightarrow 1010_2$

♦ Fractional part $\rightarrow 0.625_{10}$:

Multiply by 2 sequence:

$\times 2$	Result	Integer Part
0.625×2	1.25	1
0.25×2	0.5	0
0.5×2	1.0	1

👉 So fraction = $.101_2$

♦ Combine:

$$10.625_{10} = 1010.101_2$$

✦ Step-3: **Normalize (form: 1.fraction $\times 2^e$)**

$$1010.101_2 = 1.010101 \times 2^3$$

So:

- Fraction = 010101
- Actual exponent = 3

✦ Step-4: Biased exponent (single-precision bias = 127)

Biased exponent = $127 + 3 = 130_{10} = 10000010_2$

✦ Step-5: Mantissa (23 bits)

Normalized = 1.010101 → mantissa = 010101

Fill the rest with zeros →

010101000000000000000000

☑ Final IEEE-754 Single-Precision (32-bit)

Sign	Exponent (8-bit)	Mantissa (23-bit)
1	10000010	010101000000000000000000

👉 1 10000010 010101000000000000000000

convert 1.7 to our 8 bit floating point format

We are using the same 8-bit floating point format:

- 1 bit → Sign
- 3 bits → Exponent
- 4 bits → Mantissa
- Bias = 3

Format:

```
id="p0lmva"
| S | E E E | M M M M |
```

Step 1: Convert 1.7 to Binary

Integer part:

$$1_{10} = 1_2$$

Fractional part (multiply by 2 repeatedly):

$$0.7 \times 2 = 1.4 \rightarrow 1$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

$$0.6 \times 2 = 1.2 \rightarrow 1$$

$$0.2 \times 2 = 0.4 \rightarrow 0$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

... (continues repeating)

So,

$$1.7_{10} = 1.101100110011..._2$$

It is a **repeating binary fraction**.

Step 2: Normalize the Number

Already normalized:

$$1.1011001100..._2 \times 2^0$$

So,

- Exponent = 0
- Mantissa $\approx 1.101100...$

Step 3: Sign Bit

The number is positive.

$$S = 0$$

Step 4: Biased Exponent

Actual exponent = 0

Bias = 3

$$\text{Biased exponent} = 0 + 3 = 3$$

Step 4: Biased Exponent

Actual exponent = 0

Bias = 3

$$\text{Biased exponent} = 0 + 3 = 3$$

3 in binary (3 bits):

011

So,

Exponent = 011

Step 5: Mantissa (4 bits only)

Normalized mantissa:

1.1011001100...

We do not store the leading 1.

Take first 4 bits after decimal:

1011

(We must truncate because only 4 bits are allowed.)

So,

Mantissa = 1011

Final 8-bit Floating Point Representation

```
</> id="0m8y5f"
```

S	EEE	MMM
0	011	1011

Final Answer:

00111011

Memory System কী?

যে কোনো microprocessor-based system (ছোট হোক বা বড়) তে অবশ্যই একটি memory system থাকে।

👉 কারণ microprocessor নিজে কোনো তথ্য সংরক্ষণ করে না।
তথ্য, নির্দেশ (instruction) এবং ডেটা সংরক্ষণের জন্য memory দরকার।

Memory এর প্রধান দুই প্রকার

1 ROM (Read Only Memory)

- শুধু পড়া যায়
- সাধারণত স্থায়ী নির্দেশ (program) সংরক্ষণ করে
- Power off হলেও ডেটা থাকে

উদাহরণ:
BIOS

2 RAM (Random Access Memory)

- পড়া ও লেখা দুইটাই করা যায়
 - অস্থায়ী ডেটা সংরক্ষণ করে
 - Power off করলে ডেটা মুছে যায়
-

Memory Pin Connections

প্রতিটি memory chip-এ কিছু সাধারণ pin থাকে। এগুলো হলো:

1. Address input
2. Data input/output
3. Selection input
4. Control input

1 Address Connections

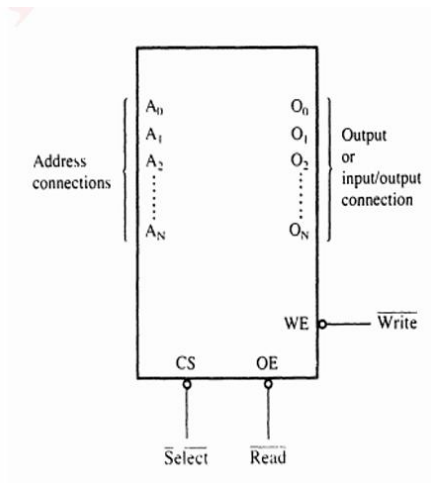
- Address pin ব্যবহার হয় নির্দিষ্ট memory location নির্বাচন করতে
- সাধারণত A0, A1, A2 ... A_n নামে লেখা থাকে
- A0 হলো least significant bit (LSB)

উদাহরণ

1K memory = 1024 location

$$1024 = 2^{10}$$

👉 তাই 1K memory device এ 10টা address pin থাকে
(A0 – A9)



2 Data Connections

- Data pin দিয়ে ডেটা ঢোকে বা বের হয়
- এখানে data লেখা বা পড়া হয়

উদাহরণ ১

16K × 1 memory

মানে:

- 16K location আছে
 - প্রতিটি location এ 1 bit সংরক্ষণ হয়
-

উদাহরণ ২

64K × 4 memory

মানে:

- 64K location
- প্রতিটি location এ 4 bit

মোট capacity = $64K \times 4$
= 256K bit

তাই একে 256K device বলা হয়।

3 Selection Connections

- প্রতিটি memory chip এ একটি select pin থাকে
- একে বলা হয়:
 - CS (Chip Select)
 - CE (Chip Enable)
 - S (Select)

👉 এই pin active না হলে memory কাজ করবে না।

কারণ একাধিক memory chip থাকলে এক সময় একটাকেই select করতে হয়।

4 Control Connections

Control pin নির্ধারণ করে memory পড়া হবে না লেখা হবে।

ROM এর Control Connection

ROM এ সাধারণত থাকে:

OE (Output Enable) বা G (Gate)

- OE active হলে ডেটা output pin দিয়ে বের হয়
- OE inactive হলে output high-impedance অবস্থায় থাকে

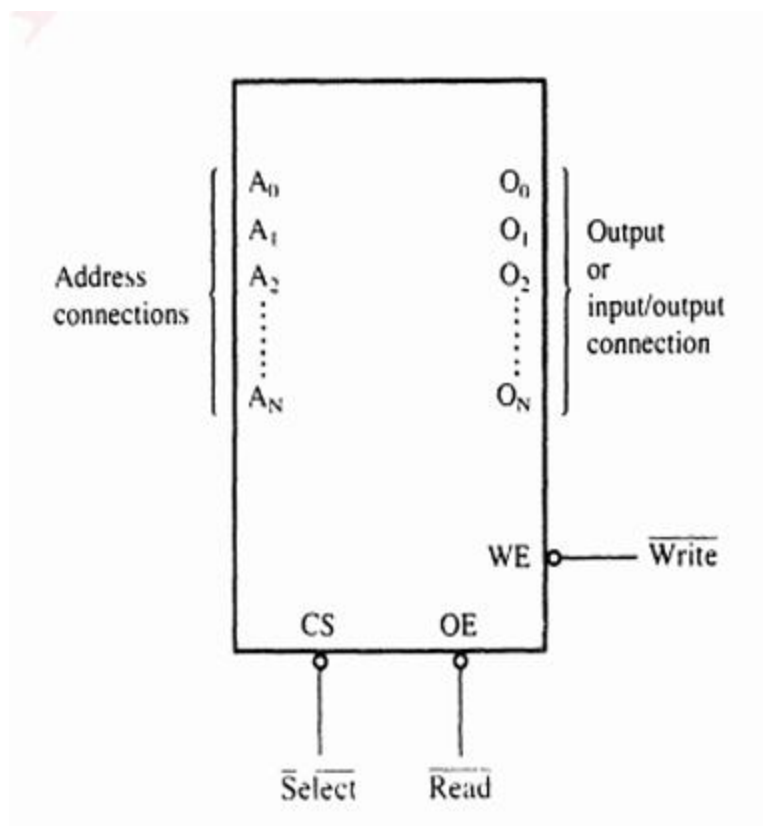
👉 ROM এ লেখা যায় না, শুধু পড়া যায়।

RAM এর Control Connection

RAM এ সাধারণত ১ বা ২টা control pin থাকে।

Case 1: একটি control pin (R/W)

- R/W = Read / Write
 - যদি:
 - R/W = 1 → Read
 - R/W = 0 → Write
 - তবে আগে CS active থাকতে হবে
-



Case 2: দুটি control pin (WE ও OE)

- WE = Write Enable
- OE = Output Enable

👉 Write করতে:

- WE active
- OE inactive

👉 Read করতে:

- OE active
- WE inactive

⚠️ খুব গুরুত্বপূর্ণ:

WE এবং OE কখনো একসাথে active হওয়া যাবে না।
এতে data conflict হতে পারে।

Address Decoding কী?

Microprocessor যখন কোনো memory location এ ডেটা পড়তে বা লিখতে চায়, তখন সে একটি address পাঠায়।

কিন্তু যদি একাধিক memory chip থাকে, তাহলে কিভাবে বোঝা যাবে কোন chip কাজ করবে?

👉 এই কাজটাই করে Address Decoder।

সংজ্ঞা (Exam-এর জন্য)

Address decoding হলো এমন একটি প্রক্রিয়া যার মাধ্যমে microprocessor থেকে আসা address signal বিশ্লেষণ করে নির্দিষ্ট memory device নির্বাচন করা হয়।

কেন Address Decoding দরকার?

- Microprocessor একাধিক memory chip ব্যবহার করতে পারে
- প্রতিটি chip কে memory map এর আলাদা অংশে কাজ করতে দিতে হয়
- Address decoder না থাকলে:
 - সব chip একসাথে সক্রিয় হয়ে যাবে
 - বা একটিই ব্যবহার করা যাবে

👉 তাই address decoder ছাড়া system কার্যত অকার্যকর হয়ে যাবে।

Memory Map কী?

Memory map হলো পুরো memory address space কে ভাগ করে বিভিন্ন device এর জন্য নির্দিষ্ট করা।

উদাহরণ:

- 0000H – 1FFFH → ROM
- 2000H – 7FFFH → RAM

এই ভাগ করাটাই address decoding এর মাধ্যমে হয়।

Main Memory Array Design

Memory Array Design কী?

একাধিক memory chip একসাথে সংযোগ করে বড় memory তৈরি করাকে Memory Array Design বলে।

16-bit Microprocessor এর Addressing Capacity

একটি 16-bit microprocessor সরাসরি যতগুলো address করতে পারে:

Formula:

$$2^n$$

এখানে n = address line সংখ্যা

যদি 16 address line থাকে:

$$2^{16} = 65,536 \text{ location} \\ = 64K \text{ byte}$$

👉 অর্থাৎ 16-bit microprocessor সর্বোচ্চ 64K memory address করতে পারে।

M/IO Signal

Microprocessor বুঝতে পারে সে Memory নিয়ে কাজ করছে নাকি I/O device নিয়ে।

এই কাজের জন্য থাকে M/IO signal।

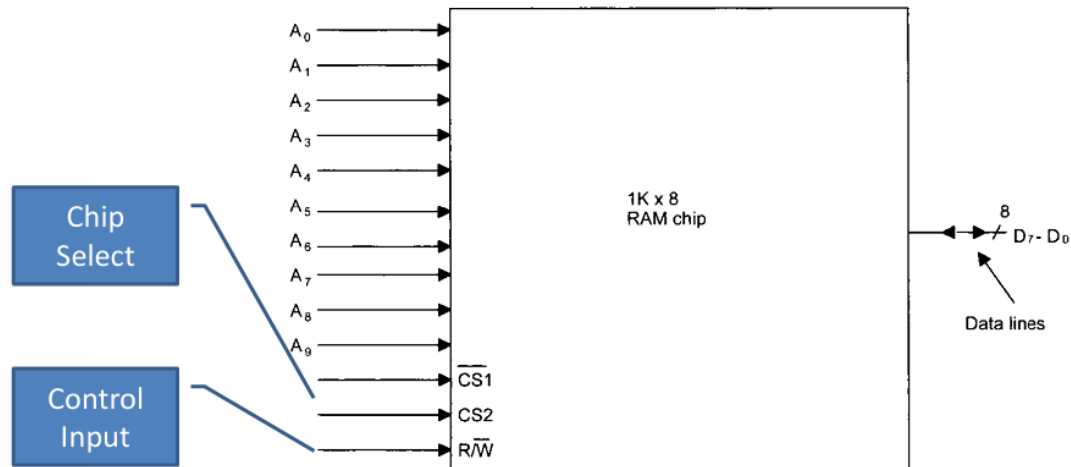
যখন M/IO = LOW

- Microprocessor I/O instruction execute করছে
 - অর্থাৎ Input/Output device নিয়ে কাজ করছে
-

যখন M/IO = HIGH

- Microprocessor Memory instruction execute করছে
- অর্থাৎ RAM বা ROM নিয়ে কাজ করছে

Main Memory Array Design



Typical 1K × 8 SRAM chip.

TABLE			Truth Table for Controlling SRAM.
$\overline{CS1}$	CS2	$R/\overline{W}$	Function
0	1	0	Write Operation
0	1	1	Read Operation
1	X	X	The chip is not selected
X	0	X	The chip is not selected

2/26/2023

RR

এটা একটি Typical 1K × 8 SRAM chip এর block diagram।

1K × 8 মানে কী?

1K × 8 এর অর্থ:

- 1K = 1024 memory location
- প্রতিটি location এ 8 bit ডেটা সংরক্ষণ হয়

অর্থাৎ মোট 1024 টি address আছে
এবং প্রতিটি address এ 1 byte (8 bit) রাখা যায়।

2 Address Lines (A0 – A9)

ছবিতে দেখা যাচ্ছে:

A0, A1, A2, ... A9

মোট 10টি address line।

কারণ:

$$1K = 1024$$

$$1024 = 2^{10}$$

👉 তাই 10টি address pin দরকার।

এই address pin গুলো দিয়ে নির্দিষ্ট memory location নির্বাচন করা হয়।

3 Data Lines (D7 – D0)

ডান পাশে আছে:

D7, D6, D5, D4, D3, D2, D1, D0

মোট 8টি data line।

কারণ:

প্রতিটি location এ 8 bit ডেটা থাকে।

👉 Write করলে এই 8 লাইনের মাধ্যমে ডেটা ঢোকে

👉 Read করলে এই 8 লাইনের মাধ্যমে ডেটা বের হয়

4 Chip Select (CS1, CS2)

ছবিতে দুইটা select pin আছে:

- CS1
- CS2

এগুলো দিয়ে নির্ধারণ করা হয় chip কাজ করবে কি না।

যদি chip select সক্রিয় না হয়, তাহলে chip কাজ করবে না।

5 Control Input (R/W)

R/W মানে:

Read / Write

- $R/W = 0 \rightarrow$ Write operation
- $R/W = 1 \rightarrow$ Read operation

কিন্তু এটা তখনই কাজ করবে যখন chip select active থাকবে।

6 Truth Table ব্যাখ্যা

নিচে দেওয়া টেবিলটা খুব গুরুত্বপূর্ণ।

CS1 CS2 R/W Function

Case 1

CS1 = 0

CS2 = 1

R/W = 0

👉 Write Operation

মানে chip select হয়েছে এবং লেখা হচ্ছে।

Case 2

CS1 = 0

CS2 = 1

R/W = 1

👉 Read Operation

মানে chip select হয়েছে এবং পড়া হচ্ছে।

Case 3

CS1 = 1

CS2 = X

R/W = X

👉 Chip select হয়নি

X মানে যেকোনো মান হতে পারে।

Case 4

CS1 = X

CS2 = 0

R/W = X

👉 Chip select হয়নি

গুরুত্বপূর্ণ বিষয়

Chip তখনই select হবে যখন:

CS1 = 0

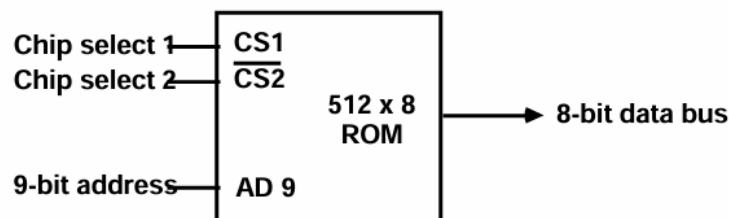
CS2 = 1

এর বাইরে যেকোনো ক্ষেত্রে chip কাজ করবে না।

Main Memory Array Design

- To connect a microprocessor to ROM/RAM chips, three address-decoding techniques are commonly used:
 - **Linear decoding**
 - **Full/Partial decoding**
 - **Memory decoding using PALs**

Typical ROM chip



2/26/2023

89

3 4K Memory বানাতে কী লাগবে?

4K = 4096 byte
 $4096 = 2^{12}$

মানে মোট 12টি address line দরকার।

কিন্তু প্রতিটি chip ব্যবহার করছে শুধু A0–A9 (10টা)
তাহলে বাকি 2টা address line (A10, A11) দিয়ে কী হবে?

👉 এই দুটো address line ব্যবহার করে কোন chip active হবে তা নির্ধারণ করা হয়।
এটাই হলো **Linear Decoding**।

4 Linear Decoding কী?

Linear decoding এ উচ্চতর address line (যেমন A10, A11) সরাসরি chip select হিসেবে ব্যবহার করা হয়।

মানে আলাদা decoder IC ব্যবহার না করে সরাসরি address bit দিয়ে chip select করা হয়।

5 4K Memory Map (ছবির টেবিল অনুযায়ী)

ধরা যাক 4টা 1K chip আছে:

Chip	Address Range (Hex)
Chip 0	0400 – 07FF
Chip 1	0800 – 0BFF
Chip 2	1000 – 13FF
Chip 3	2000 – 23FF

প্রতিটি chip 1K = 400H (Hex) জায়গা নেয়।

কারণ:

1K = 1024 decimal = 400H hex

Binary Address Pattern A15 A14 A13 A12 A11 A10 A9 A8 A7 A6 A5 A4 A3 A2 A1 A0	Device Selected	Address Assignment in Hex
0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 . 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1	RAM CHIP 0	0400 to 07FF
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 . 0 0 0 0 1 0 1 1 1 1 1 1 1 1 1 1	RAM CHIP 1	0800 to 0BFF
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 . 0 0 0 1 0 0 1 1 1 1 1 1 1 1 1 1	RAM CHIP 2	1000 to 13FF
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 . 0 0 1 0 0 0 1 1 1 1 1 1 1 1 1 1	RAM CHIP 3	2000 to 23FF

6 Binary Pattern ব্যাখ্যা

উচ্চ address bit (যেমন A11, A10) পরিবর্তনের মাধ্যমে chip নির্বাচন হয়।

ধরা যাক:

A11 A10

00 → Chip 0

01 → Chip 1

10 → Chip 2

11 → Chip 3

এভাবে প্রতিটি chip আলাদা 1K block পায়।

Main Memory Array Design

- **Linear decoding**

Suppose we have

4K SRAM chip array comprised of the four 1K SRAM chips

➤ This system can be expanded up to a total capacity of 6K using A_{14} , and A_{15} , as chip selects for two more 1K SRAM chips.

Address Range (Hex)	SRAM Chip Number
3800-3BFF	I
3400-37FF	II
2C00-2FFF	III
1C00-1FFF	IV

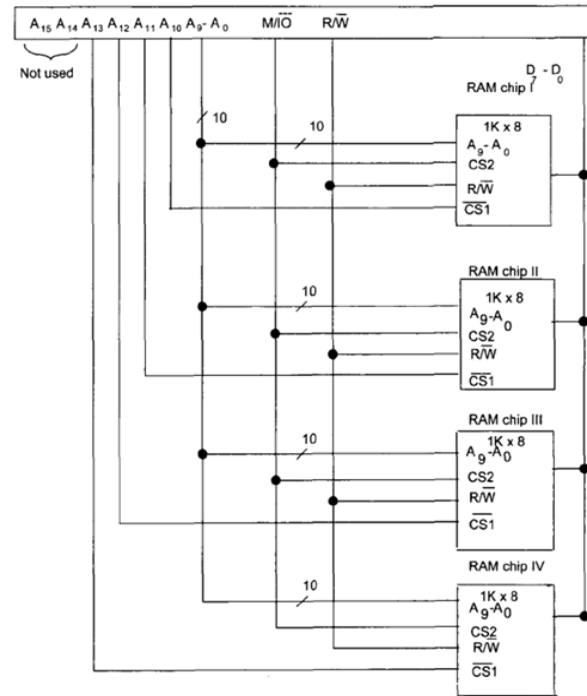


FIGURE 3.8 Microprocessor connected to 4K SRAM using the linear select decoding technique.

এই স্লাইডটিতে **Linear Decoding (বা Linear Select Decoding)** পদ্ধতিতে কীভাবে একাধিক SRAM চিপ ব্যবহার করে একটি মেইন মেমোরি অ্যারে (Main Memory Array) ডিজাইন করা হয়, তা দেখানো হয়েছে।

খুব সহজ ভাষায় পুরো বিষয়টা নিচে ভেঙে বুঝিয়ে দেওয়া হলো:

১. মূল লক্ষ্য (The Goal)

এখানে আমাদের উদ্দেশ্য হলো ১K (১ কিলোবাইট) সাইজের ৪টি SRAM চিপ দিয়ে মোট ৪K ক্ষমতার একটি মেমোরি সিস্টেম তৈরি করা।

- প্রতিটি চিপের সাইজ: $1K \times 8$ (অর্থাৎ ১০২৪টি লোকেশন, প্রতিটিতে ৮-বিট বা ১ বাইট ডাটা থাকে)।
- ১K লোকেশনকে নির্দিষ্ট করার জন্য মোট ১০টি অ্যাড্রেস লাইন (A_9 থেকে A_0) প্রয়োজন ($2^{10} = 1024$)।

২. লিনিয়ার ডিকোডিং (Linear Decoding) কী?

সাধারণত অনেকগুলো মেমোরি চিপ সিলেক্ট করার জন্য আমরা Decoder IC (যেমন 2-to-4 বা 3-to-8 decoder) ব্যবহার করি। কিন্তু Linear Decoding পদ্ধতিতে কোনো এক্সট্রা ডিকোডার চিপ ব্যবহার না করে, প্রসেসরের অব্যবহৃত হায়ার-অর্ডার অ্যাড্রেস লাইনগুলোকে সরাসরি চিপের Chip Select ($\overline{CS1}$) পিনের সাথে জুড়ে দেওয়া হয়।

এখানে ডায়াগ্রামে খেয়াল করুন:

- $A_9 - A_0$ লাইনগুলো ৪টি চিপেরই অ্যাড্রেস পিনে সমান্তরালে (Parallel) কানেক্টেড।
- বাকি চারটা লাইন ($A_{13}, A_{12}, A_{11}, A_{10}$) দিয়ে আলাদা আলাদাভাবে ৪টি চিপকে অন/অফ (Select) করা হচ্ছে:
 - A_{10} যুক্ত আছে RAM chip I এর $\overline{CS1}$ (Active Low) পিনের সাথে।
 - A_{11} যুক্ত আছে RAM chip II এর $\overline{CS1}$ পিনের সাথে।
 - A_{12} যুক্ত আছে RAM chip III এর $\overline{CS1}$ পিনের সাথে।
 - A_{13} যুক্ত আছে RAM chip IV এর $\overline{CS1}$ পিনের সাথে।

💡 নোট: যেহেতু $\overline{CS1}$ এর ওপর বার ($\overline{CS}$) আছে, তাই যে লাইনে ০ (Low) সিগন্যাল আসবে, সেই চিপটি সচল বা সিলেক্ট হবে। বাকি চিপগুলোর লাইনে ১ থাকতে হবে।

৩. অ্যাড্রেস রেঞ্জ কীভাবে বের হলো? (Address Range Table)

বামে দেওয়া টেবিলটিতে প্রতিটি চিপের Hexadecimal অ্যাড্রেস রেঞ্জ দেওয়া আছে। চলুন বাইনারি দিয়ে একটি মিলিয়ে দেখি:

- **RAM Chip I (3800 - 3BFF):**
 - Hex 3800 কে বাইনারি করলে হয়: 0011 1000 0000 0000
 - পজিশন অনুযায়ী:
 - $A_{15} = 0, A_{14} = 0$
 - $A_{13} = 1, A_{12} = 1, A_{11} = 1$
 - $A_{10} = 0 \rightarrow$ (এটি ০ হওয়ায় শুধুমাত্র Chip I সিলেক্ট হবে)।

অনুরূপভাবে:

- যখন $A_{11} = 0$ হয়, তখন Chip II সিলেক্ট হয় (Range: 3400-37FF)।
- যখন $A_{12} = 0$ হয়, তখন Chip III সিলেক্ট হয় (Range: 2C00-2FFF)।
- যখন $A_{13} = 0$ হয়, তখন Chip IV সিলেক্ট হয় (Range: 1C00-1FFF)।

8. অন্যান্য সিগন্যাল

- CS2 (Active High): এটি সব চিপেই $M/\overline{IO}$ (Memory/IO) লাইনের সাথে যুক্ত। যখন প্রসেসর মেমোরি অপারেশন করতে চায়, তখন এই লাইনটি 1 হয় এবং সব চিপের CS2 ইনপুট সচল হয়।
- $R/\overline{W}$: এটি রিড (Read) বা রাইট (Write) অপারেশন নিয়ন্ত্রণ করে।
- A_{14}, A_{15} : এই দুটি লাইন বর্তমানে অব্যবহৃত ("Not used")। স্লাইডের টেক্সটে বলা আছে, ভবিষ্যতে যদি মেমোরি আরও বাড়িয়ে ৬K করতে হয়, তবে এই দুটি লাইন ব্যবহার করে আরও ২টি ১K চিপ যুক্ত করা যাবে।

লিনিয়ার ডিকোডিংয়ের সুবিধা ও অসুবিধা:

- **সুবিধা:** সার্কিট খুব সহজ হয় এবং কোনো অতিরিক্ত ডিকোডার আইসি (Decoder IC) লাগে না, ফলে খরচ কমে।
- **অসুবিধা:** এতে অনেক অ্যাড্রেস লাইন অপচয় হয় (Bus wastage) এবং এতে **Bus Conflict** হওয়ার ঝুঁকি থাকে (যদি ভুলবশত একসাথে দুটি চিপের লাইনে 0 চলে আসে)।

Main Memory Array Design

কম্পিউটারের **মেইন মেমোরি** হলো র‍্যান্ডম অ্যাকসেস মেমোরি (RAM), যা ঠিকমতো অ্যাড্রেস করার জন্য ঠিকভাবে ডিজাইন করা লাগে। এখানে আমরা **Address Decoding** নিয়ে আলোচনা করছি।

1. Linear Decoding (লাইনিয়ার ডিকোডিং)

উপকারিতা (Advantage)

- এতে **ডিকোডিং হার্ডওয়্যার দরকার হয় না**। অর্থাৎ আমরা সরাসরি এড্রেস লাইন ব্যবহার করে মেমোরি চিপ নির্বাচন করতে পারি।

অসুবিধা (Disadvantage)

1. Bus Conflict (বাস কনফ্লিক্ট)

- যদি একাধিক এড্রেস লাইন একসাথে লো থাকে (যেমন: A10–A13 লাইন), তাহলে একাধিক SRAM চিপ একই সময়ে নির্বাচিত হবে।
- **সমাধান:** সফটওয়্যার এমনভাবে লেখা দরকার যাতে কখনও এমন এড্রেসে রিড বা রাইট না হয় যেখানে একাধিক চিপ একসাথে নির্বাচিত হবে।

2. Memory Foldback (মেমোরি ফোল্ডব্যাক)

- একই SRAM চিপের জন্য অনেক এড্রেস ব্যবহার হয়, ফলে অনেক এড্রেস স্পেস বাঁচে না। উদাহরণ: ঠিক তখনই SRAM chip 1 নির্বাচিত হয় যখন এড্রেস B800 বা 3800।
 - **সমাধান: Full Decoded Memory Addressing** ব্যবহার করা। অর্থাৎ, প্রতিটি মেমোরি লোকেশনকে আলাদা এড্রেস দেওয়া।
-

2. Address Decoding Strategy (এড্রেস ডিকোডিং কৌশল)

মেমোরির পুরো স্পেস বাস্তবে প্রয়োগ করা হবে কি না, তার ওপর নির্ভর করে দুটি কৌশল আছে:

a) Full Address Decoding (পূর্ণ এড্রেস ডিকোডিং)

- সব এড্রেস লাইন ব্যবহার করে মেমোরি লোকেশন নির্ধারণ করা হয়।
- প্রতিটি ফিজিক্যাল মেমোরি লোকেশন আলাদা এড্রেস দ্বারা চিহ্নিত হয়।
- সুবিধা: কোন Bus Conflict বা Foldback হয় না।

b) Partial Address Decoding (আংশিক এড্রেস ডিকোডিং)

- সব এড্রেস স্পেস বাস্তবে ব্যবহার করা হয় না। তাই শুধুমাত্র কিছু এড্রেস লাইন ব্যবহার করা হয়।
 - একাধিক এড্রেস একই ফিজিক্যাল মেমোরি লোকেশন নির্দেশ করতে পারে। (যে এড্রেস লাইন ব্যবহার হয়নি তাদের সব কস্ট্রিনেশন ব্যবহার হবে)
 - অর্থাৎ, একটি SRAM চিপ অনেক এড্রেসের সাথে ম্যাপ হতে পারে।
 - অসুবিধা: কিছু এড্রেস স্পেস ফাঁকা বা বর্জিত হয়ে যায়, এবং Bus Conflict হতে পারে।
-

সহজ উদাহরণ

ধরে নাও আমাদের 4KB মেমোরি আছে, 2KB SRAM চিপ।

- **Linear/Partial decoding:**
 - 2KB SRAM চিপ দুটি ভিন্ন এড্রেসে নির্বাচিত হবে (Foldback)।
 - যদি ভুল এড্রেসে রিড/রাইট করি → Bus Conflict।
 - **Full decoding:**
 - প্রতিটি চিপ আলাদা এড্রেস পাবে।
 - কোনো Bus Conflict বা Waste হবে না।
-
-

1 Decoder কী?

Decoder হলো একটি **combinational logic circuit**।

এর বৈশিষ্ট্য:

- n টি input থাকে
- সর্বোচ্চ 2^n টি output থাকে
- Enable সক্রিয় থাকলে ($E = 1$)
ইনপুটের নির্দিষ্ট কম্বিনেশনের জন্য একটি মাত্র output High (1) হবে

অর্থাৎ Decoder একটি নির্দিষ্ট binary code শনাক্ত করে।

এটা আসলে input ভেরিয়েবলগুলোর **minterm generate** করে।

2 2-to-4 Decoder

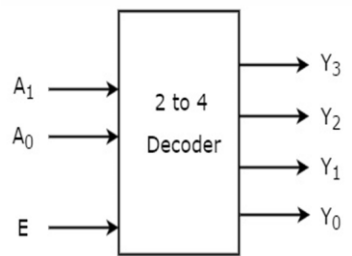
এখানে:

- Input: A1, A0
- Enable: E
- Output: Y3, Y2, Y1, Y0

যেহেতু 2টি input আছে $\rightarrow 2^2 = 4$ টি output হবে।

3 Truth Table (Enable সহ)

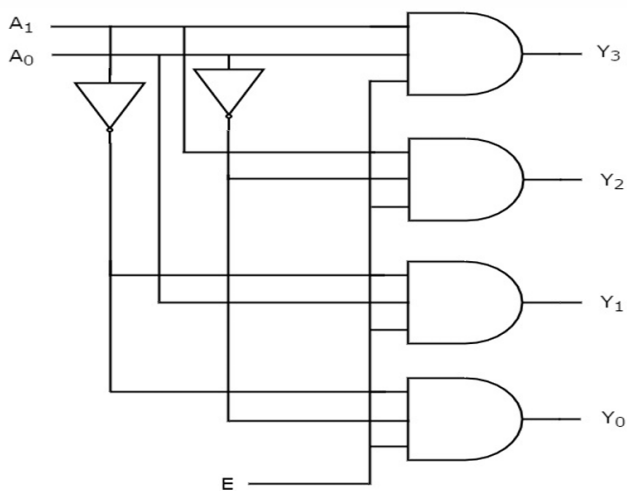
E	A1	A0	Y3	Y2	Y1	Y0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0



ব্যখ্যা:

- $E = 0$ হলে $\rightarrow$ সব output 0
- $E = 1$ হলে $\rightarrow$ input অনুযায়ী একটি মাত্র output = 1

Decoder



4 Boolean Expression

$$Y_0 = E \cdot A_1 \bar{} \cdot A_0 \bar{}$$

$$Y_1 = E \cdot A_1 \bar{} \cdot A_0$$

$$Y_2 = E \cdot A_1 \cdot A_0 \bar{}$$

$$Y_3 = E \cdot A_1 \cdot A_0$$

এগুলোই 2টি input এর 4টি minterm।

5 Memory Design এ Decoder কেন দরকার?

Linear decoding এ সরাসরি address line ব্যবহার করা হয়।

কিন্তু যদি multiple memory chip থাকে, তাহলে:

- উচ্চ address bit (যেমন A15–A13)
- Decoder এ দেওয়া হয়
- Decoder এর output → Chip Select হিসেবে ব্যবহৃত হয়

উদাহরণ:

যদি 3টি address line থাকে → 3-to-8 decoder ব্যবহার করে 8টি chip select করা যায়।

এতে address space overlap হয় না।

6 সহজভাবে মনে রাখার উপায়

Decoder কী করে?

👉 Binary number → One active output

যেমন:

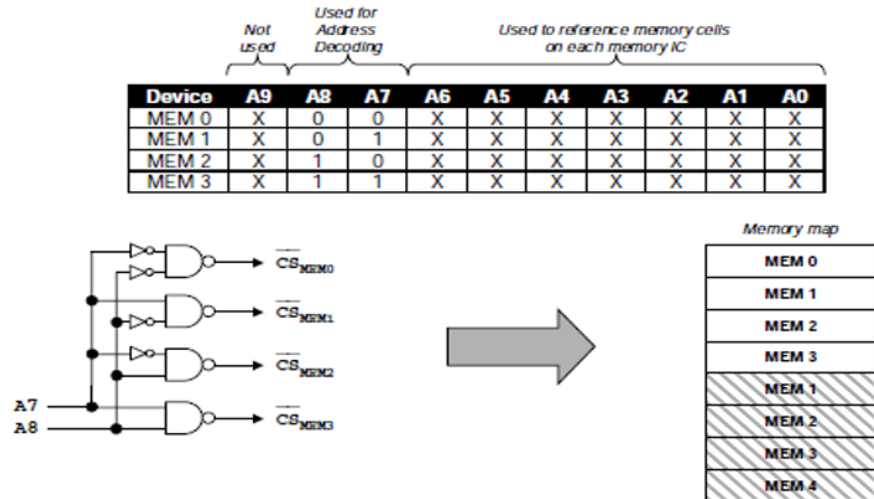
Input = 10 → Output Y2 = 1

এটা অনেকটা binary to line selector।

চল, Partial আর Full address decoding একই উদাহরণ দিয়ে পরীক্ষারভাবে বুঝে নিই।

Partial Address Decoding (আংশিক অ্যাড্রেস ডিকোডিং)

SOLUTION



5/2023

পুরো সমস্যা এবং সমাধানটি নিচে সহজ ভাষায় ধাপে ধাপে বুঝিয়ে দেওয়া হলো:

১. সমস্যা বা শর্তাবলি (The Problem Context)

- মাইক্রোপ্রসেসরের মোট অ্যাড্রেস লাইন: ১০টি (A_9 থেকে A_0)। এর মানে প্রসেসরটি সর্বোচ্চ $2^{10} = 1024$ বাইট বা ১ KB মেমোরি হ্যান্ডেল করতে পারে।
- আমাদের দরকার (Target): মাত্র ৫১২ বাইট মেমোরি ইমপ্লিমেন্ট করা।
- উপলব্ধ মেমোরি চিপ: প্রতিটি চিপের সাইজ ১২৮ বাইট।
 - ৫১২ বাইট বানাতে আমাদের মোট চিপ লাগবে: $\frac{512}{128} = 4$ টি চিপ (MEM 0, MEM 1, MEM 2, MEM 3)।
 - ১২৮ বাইটের প্রতিটি চিপের ভেতরের মেমোরি সেলগুলোকে নির্দিষ্ট করার জন্য লাইন লাগে $2^7 = 128$, অর্থাৎ ৭টি লাইন (A_6 থেকে A_0)।
- বিশেষ শর্ত: মেমোরি ম্যাপের Upper Half (উপরের অর্ধেকাংশে) এই ফিজিক্যাল মেমোরি স্থাপন করতে হবে।

২. সমাধান ও অ্যাড্রেস ডিস্ট্রিবিউশন টেবিল

১০টি লাইনের ($A_9 - A_0$) কাজকে ৩টি ভাগে ভাগ করা হয়েছে:

- A_6 থেকে A_0 (Used to reference memory cells): এই ৭টি লাইন সরাসরি প্রতিটি ১২৮-বাইটের চিপের ভেতরকার লোকেশন ট্র্যাক করতে ব্যবহৃত হবে।
- A_8 এবং A_7 (Used for Address Decoding): এই ২টি লাইন ব্যবহার করে ৪টি চিপকে আলাদাভাবে সিলেক্ট করা হবে ($2^2 = 4$ টি কম্বিনেশন)।
- A_9 (Not used): শর্ত অনুযায়ী মেমোরি ম্যাপের Upper Half-এ ডিজাইন করতে বলা হয়েছে। কিন্তু আংশিক বা Partial ডিকোডিংয়ের নিয়মই হলো—কিছু লাইনকে আমরা ডিকোডিং সার্কিটে পাঠাবো না, সেগুলোকে 'Don't Care' বা $\times$ হিসেবে রেখে দেবো। এখানে A_9 এবং $A_6 - A_0$ এর মধ্যবর্তী কোনো লাইন (যেমন A_6 টেবিলে ডিকোডিংয়ে বাদ পড়েছে) ডিকোডিং লজিকে যুক্ত করা হয়নি।

টেবিলে ডিকোডিং লজিক (A_8, A_7):

- MEM 0: $A_8 = 0, A_7 = 0$
- MEM 1: $A_8 = 0, A_7 = 1$
- MEM 2: $A_8 = 1, A_7 = 0$
- MEM 3: $A_8 = 1, A_7 = 1$

৩. লজিক সার্কিট ডায়াগ্রাম

নিচের গেটগুলোর কানেকশন খেয়াল করুন:

- চিপগুলোর সিলেক্ট পিন হচ্ছে Active Low ($\overline{CS}$), অর্থাৎ ০ পেলে চিপ অন হবে। তাই এখানে NAND Gate এবং ইনভার্টার ব্যবহার করা হয়েছে।
- উদাহরণস্বরূপ, যখন $A_8 = 0$ এবং $A_7 = 0$ হবে, তখন প্রথম NAND গেটের দুটি ইনপুটই ইনভার্টারের কারণে ১ এবং ১ পাবে। ফলে এর আউটপুট হবে ০ এবং শুধুমাত্র $\overline{CS}_{MEM0}$ চিপটি অ্যাক্টিভ হবে।

8. মেমোরি ম্যাপ এবং "Memory Shadowing" (ডানপাশের চিত্র)

Partial Decoding-এর সবচেয়ে বড় বৈশিষ্ট্য বা অসুবিধা হলো Memory Shadowing বা Aliasing।

যেহেতু A_9 লাইনটিকে ডিকোডিং সার্কিটে কানেক্ট করা হয়নি (টেবিলে $\times$ বা Don't care), তাই প্রসেসর A_9 এর মান 0 রাখুক আর 1 রাখুক, ডিকোডিং সার্কিট তা টের পাবে না।

- এর ফলে, একই ফিজিক্যাল মেমোরি চিপ (যেমন MEM 0) মেমোরি ম্যাপের দুটি ভিন্ন ভিন্ন অ্যাড্রেসে দেখা যাবে!
- ডায়াগ্রামের নিচের অংশে দাগ টানা (shaded) যে MEM 0, MEM 1, MEM 2, MEM 3 দেখা যাচ্ছে, সেগুলো আসলে নতুন কোনো মেমোরি চিপ নয়। ওগুলো হলো আসল চিপগুলোরই "Mirror" বা "Shadow" image।

💡 সহজ কথায়: $A_9 = 0$ হলে প্রসেসর মেমোরি ম্যাপের এক জায়গায় ডেটা রিড/রাইট করবে, আবার $A_9 = 1$ হলেও ঠিক একই ফিজিক্যাল চিপেরই ডেটা রিড/রাইট হবে। মেমোরি লোকেশনের এই ডুপ্লিকেট রূপকেই Shaded Area দিয়ে মেমোরি ম্যাপে দেখানো হয়েছে।

মেমোরি ম্যাপের Shaded Area বা Shadowing কী?

প্রসেসরের নিজের হিসাব অনুযায়ী (মেমোরি ম্যাপে):

- $A_9 = 0$ থাকা অবস্থায় একটা মেমোরি লোকেশন আছে (যেটাকে আমরা আসল মেমোরি বলছি)।
- $A_9 = 1$ থাকা অবস্থায় তার কাছে মনে হচ্ছে সম্পূর্ণ আলাদা আরেকটা মেমোরি লোকেশন আছে (যেটাকে চিত্রে দাগ টানা বা Shaded MEM 0 দেখানো হয়েছে)।

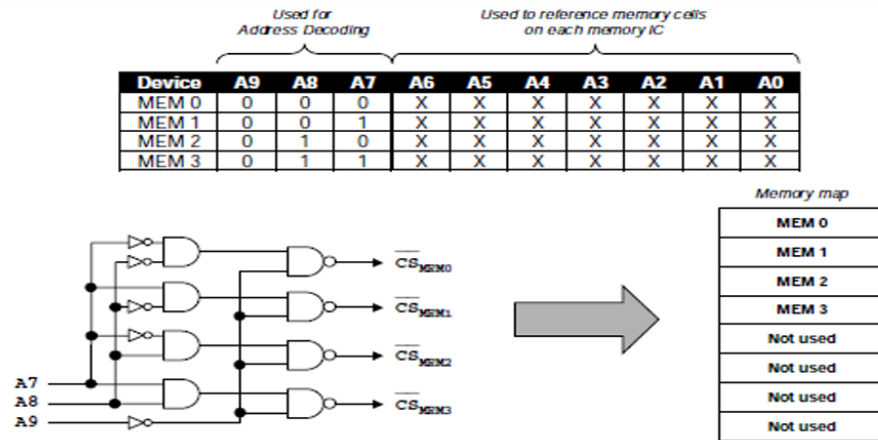
কিন্তু বাস্তবে চিপ কিন্তু একটাই! প্রসেসর ভাবছে সে দুটি ভিন্ন জায়গায় ডেটা রাখছে, কিন্তু আসলে সে ঘুরেফিরে একই চিপের ওপর ডেটা ওভাররাইট (Overwrite) করছে।

মেমোরির এই অবিকল নকল বা ডুপ্লিকেট রূপ তৈরি হওয়াকেই বলা হয় Memory Shadowing বা Aliasing। চিত্রে দাগ টানা অংশগুলো দিয়ে এটাই বোঝানো হয়েছে যে, এগুলো কোনো নতুন চিপ নয়, বরং ওপরের আসল ৪টি চিপেরই "ছায়া" বা Mirror Image।

Full Address Decoding

Full address decoding

- Let's assume the same microprocessor with 10 address lines (1KB memory)
 - However, this time we wish to implement only 512 bytes of memory
 - We still must use 128-byte memory chips
 - Physical memory must be placed on the upper half of the memory map
- SOLUTION**



১. প্রশ্ন বা শর্তগুলো আরেকবার দেখে নিই

- মোট অ্যাড্রেস লাইন: ১০টি (A_9 থেকে A_0)। অর্থাৎ প্রসেসরের মোট মেমোরি ক্ষমতা ১ KB।
- আমাদের দরকার: মাত্র ৫১২ বাইট মেমোরি (যার জন্য ১২৮-বাইটের ৪টি চিপ লাগবে: MEM 0, MEM 1, MEM 2, MEM 3)।
- চিপের ভেতরের জন্য লাইন: ১২৮ বাইটের জন্য লাগবে ৭টি লাইন (A_6 থেকে A_0)।
- শর্ত (খুব গুরুত্বপূর্ণ): স্লাইডের টেক্সটে বলা আছে—“Physical memory must be placed on the upper half of the memory map”। অর্থাৎ মেমোরি ম্যাপের ওপরের অংশে চিপগুলো বসাতে হবে।

⚠ এখানে স্লাইড মেকারের আরেকটি ভুল (Typo): > স্লাইডের টেক্সটে Upper Half (ওপরের অর্ধেক) বলা হলেও, তারা টেবিলে $A_9 = 0$ ধরে ডিজাইন করেছে। $A_9 = 0$ হওয়া মানে কিন্তু মেমোরি ম্যাপের Lower Half (নিচের অর্ধেক) সিলেক্ট হওয়া। আমরা স্লাইডের টেবিল ও ডায়াগ্রাম অনুযায়ী নিচে পুরোটা বুঝবো।

২. ফুল ডিকোডিং (Full Decoding) কী এবং কীভাবে কাজ করছে?

ফুল ডিকোডিংয়ের মূল কথা হলো—চিপের নিজস্ব ৭টি লাইন বাদে প্রসেসরের বাকি যতগুলো অ্যাড্রেস লাইন থাকবে, তাদের প্রত্যেকটিকে (None left behind) ডিকোডিং সার্কিটে ব্যবহার করতে হবে। কোনো লাইনকে 'Don't Care' বা $\times$ রাখা যাবে না।

এখানে চিপের বাইরের লাইন বাকি আছে ৩টি: A_9, A_8, A_7 । টেবিলে খেয়াল করো, এই ৩টি লাইনকেই ডিকোডিংয়ের জন্য নির্দিষ্ট মান দেওয়া হয়েছে:

- A_9 এর ভূমিকা: সব চিপের জন্যই $A_9 = 0$ করে দেওয়া হয়েছে। এর মানে হলো প্রসেসরের A_9 লাইনটি যখনই ০ হবে, কেবল তখনই মেমোরি চিপগুলো সচল হওয়ার সুযোগ পাবে। যদি $A_9 = 1$ হয়, তবে কোনো চিপই সিলেক্ট হবে না!
- A_8 এবং A_7 এর ভূমিকা: এই দুটি লাইন আগের মতোই ৪টি চিপকে আলাদা করতে ব্যবহৃত হচ্ছে:
 - MEM 0: $A_9 = 0, A_8 = 0, A_7 = 0$
 - MEM 1: $A_9 = 0, A_8 = 0, A_7 = 1$
 - MEM 2: $A_9 = 0, A_8 = 1, A_7 = 0$
 - MEM 3: $A_9 = 0, A_8 = 1, A_7 = 1$

৩. লজিক সার্কিট ডায়াগ্রাম

সার্কিটটি লক্ষ্য করো, আগের চেয়ে এখানে গেটের সংখ্যা বেড়ে গেছে:

1. বাম পাশের প্রথম সারির NAND গেটগুলো দিয়ে প্রথমে A_8 এবং A_7 এর কম্বিনেশন চেক করা হচ্ছে।
2. এরপর ডান পাশে আরেকটি করে NAND গেট বসানো হয়েছে। এই গেটগুলোর একটি ইনপুট আসছে প্রথম গেট থেকে, আর অন্য ইনপুটটি সরাসরি যুক্ত হয়েছে A_9 লাইনের ইনভার্টারের ($\overline{A_9}$) সাথে।
3. যেহেতু $A_9 = 0$, তাই ইনভার্টারের পর সেটি 1 হয়ে ডানপাশের সব গেটে পৌঁছায়। ফলে যখনই প্রসেসর $A_9 = 0$ পাঠাবে এবং A_8, A_7 এর নির্দিষ্ট মান দেবে, ঠিক তখনই কেবল নির্দিষ্ট চিপটির $\overline{CS}$ পিন ০ (Active) হবে।

৪. মেমোরি ম্যাপ (ডানপাশের চিত্র)

এবার মেমোরি ম্যাপের দিকে তাকাও, জাদুকরী পরিবর্তনটা এখানেই:

- ম্যাপের ওপরের অর্ধেক অংশে (MEM 0 , MEM 1 , MEM 2 , MEM 3) আমাদের ৪টি আসল চিপ সুন্দরভাবে বসে গেছে। (এটি মূলত $A_9 = 0$ এর জন্য বরাদ্দ)।
- ম্যাপের নিচের অর্ধেক অংশে (যখন $A_9 = 1$) সেখানে আর কোনো দাগ টানা বা Shadow মেমোরি নেই! সেখানে স্পষ্ট লেখা আছে "Not used"।

💡 একদম সহজ সারসংক্ষেপ:

যেহেতু আমরা A_9 লাইনটিকে ডিকোডিং সার্কিটে শক্ত করে বেঁধে দিয়েছি ($A_9 = 0$), তাই প্রসেসর যদি ভুল করেও $A_9 = 1$ করে মেমোরি খুঁজতে আসে, ডিকোডিং সার্কিট সাথে সাথে চিপগুলোকে লক করে দেবে। ফলে আগের বারের মতো কোনো "নকল বা ছায়া (Shadow)" মেমোরি তৈরি হওয়ার সুযোগই নেই। মেমোরির বাকি অংশ খালিই (Not used) পড়ে থাকবে।

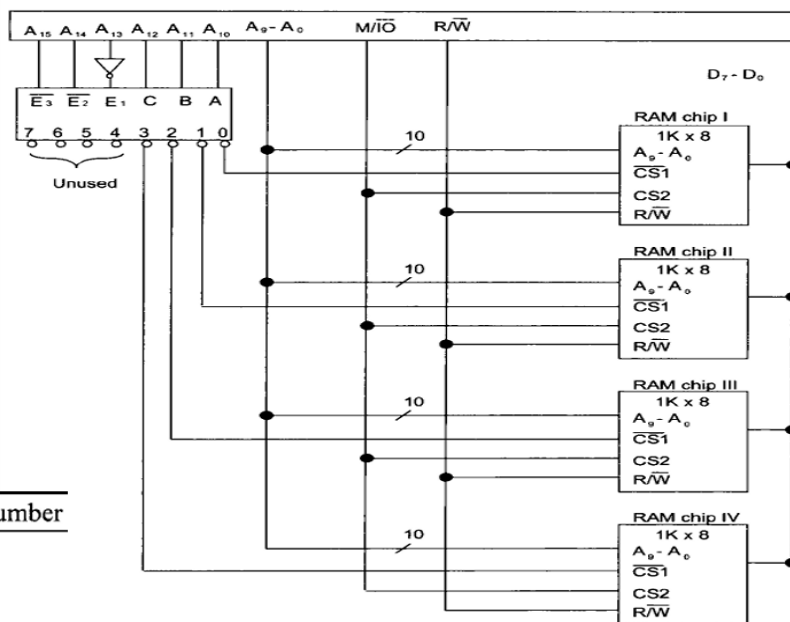
Partial vs Full Decoding

বিষয়	Partial	Full
Hardware	কম	বেশি
Circuit	সহজ	জটিল
Address Overlap	হতে পারে	হয় না
Reliable	কম	বেশি

• Full Decoding.

- Using 3x8 decoder output selects one of the four 1K SRAM chips, depending on the values of A_{12} , A_{11} , and A_{10}
- Decoder output enabled only when $E_3 = E_2 = 0$ and $E_1 = 1$

A_{12}	A_{11}	A_{10}	SRAM Chip Number
0	0	0	I
0	0	1	II
0	1	0	III
0	1	1	IV



Interconnecting a microprocessor with a 4K RAM using full decoded memory addressing.

প্রথম স্লাইডে আমরা যে ৪K মেমোরি সিস্টেমটি লিনিয়ার ডিকোডিং দিয়ে দেখেছিলাম (যেখানে ৪টি আলাদা অ্যাড্রেস লাইন অপচয় হয়েছিল), ঠিক সেই ৪K সিস্টেমটিকেই এখানে একটি ডেডিকেটেড ডিকোডারের মাধ্যমে নিখুঁতভাবে ডিজাইন করা হয়েছে।

চলো একদম সহজ ভাষায় সার্কিটের খুঁটিনাটি বুঝে নিই:

১. মূল লক্ষ্য এবং উপাদান (The Components)

- উদ্দেশ্য: ৪টি $1K \times 8$ SRAM চিপ দিয়ে মোট ৪K মেমোরি তৈরি করা।
- চিপের ভেতরের অ্যাড্রেস লাইন: ১K মেমোরির জন্য যথারীতি ১০টি লাইন ($A_9 - A_0$) সরাসরি চিপগুলোতে চলে গেছে।
- ডিকোডার আইসি: এখানে একটি 3×8 Decoder (যেমন 74LS138 IC) ব্যবহার করা হয়েছে। এই ডিকোডারের কাজ হলো ৩টি ইনপুট লাইন দেখে ৮টি আউটপুট লাইনের (০ থেকে ৭) যেকোনো একটিকে সিলেক্ট করা।

২. ডিকোডারের ইনপুট কানেকশন (C, B, A)

ডিকোডারের প্রধান ৩টি ইনপুট পিন হলো C, B, A । এদের সাথে প্রসেসরের ৩টি অ্যাড্রেস লাইন যুক্ত করা হয়েছে:

- $C = A_{12}$
- $B = A_{11}$
- $A = A_{10}$

বাম পাশের নিচের টেবিলটি খেয়াল করো। এই ৩টি লাইনের বাইনারি কম্বিনেশনের ওপর ভিত্তি করে ডিকোডারের আউটপুট পিনগুলো সচল হবে এবং চিপ সিলেক্ট করবে:

- $A_{12}A_{11}A_{10} = 000$ হলে ডিকোডারের ০ নং পিন অ্যাক্টিভ হবে → এটি যুক্ত আছে RAM chip I এর $\overline{CS1}$ এর সাথে।
- $A_{12}A_{11}A_{10} = 001$ হলে ডিকোডারের ১ নং পিন অ্যাক্টিভ হবে → এটি যুক্ত আছে RAM chip II এর $\overline{CS1}$ এর সাথে।
- $A_{12}A_{11}A_{10} = 010$ হলে ডিকোডারের ২ নং পিন অ্যাক্টিভ হবে → এটি যুক্ত আছে RAM chip III এর $\overline{CS1}$ এর সাথে।
- $A_{12}A_{11}A_{10} = 011$ হলে ডিকোডারের ৩ নং পিন অ্যাক্টিভ হবে → এটি যুক্ত আছে RAM chip IV এর $\overline{CS1}$ এর সাথে।

💡 ডিকোডারের বাকি ৪টি আউটপুট পিন (4, 5, 6, 7) বর্তমানে খালি বা অব্যবহৃত (Unused) অবস্থায় আছে। ভবিষ্যতে মেমোরি আরও ৪K বাড়াতে চাইলে সেখানে আরও ৪টি চিপ জোড়া যাবে।

৩. ডিকোডারের এনাবল পিনগুলোর খেল ($\overline{E3}$, $\overline{E2}$, $E1$)

"Full Decoding" নিশ্চিত করার জন্য প্রসেসরের বাকি থাকা হায়ার-অর্ডার অ্যাড্রেস লাইনগুলোকে (A_{15} , A_{14} , A_{13}) ডিকোডারের Enable পিনগুলোর সাথে বেঁধে দেওয়া হয়েছে।

একটি ডিকোডার তখনই কাজ করে যখন তার সবকটি এনাবল কন্ডিশন পূরণ হয়। এই ডিকোডারের নিয়ম হলো:

- $\overline{E3} = 0$ হতে হবে (Active Low)
- $\overline{E2} = 0$ হতে হবে (Active Low)
- $E1 = 1$ হতে হবে (Active High)

ডায়াগ্রামের ওপরের কানেকশনটি লক্ষ্য করো:

- A_{15} যুক্ত আছে $\overline{E3}$ এর সাথে। (কাজেই চিপ অন হতে হলে A_{15} কে অবশ্যই 0 হতে হবে)।
- A_{14} যুক্ত আছে $\overline{E2}$ এর সাথে। (কাজেই A_{14} কে অবশ্যই 0 হতে হবে)।
- A_{13} লাইনে একটি Inverter (NOT Gate) বসিয়ে তা যুক্ত করা হয়েছে $E1$ এর সাথে। (যেহেতু $E1$ কে 1 হতে হবে, তাই ইনভার্টারের আগের A_{13} লাইনটিকে অবশ্যই 0 হতে হবে, যাতে ইনভার্ট হয়ে সেটি 1 হয়)।

🔴 তার মানে শর্ত হলো: প্রসেসরের A_{15} , A_{14} , A_{13} এই ৩টি লাইন যখন একসাথে 0, 0, 0 হবে, কেবল তখনই ডিকোডার আইসিটি নিজে "অন" হবে এবং মেমোরি চিপ সিলেক্ট করার অনুমতি দেবে। অন্য যেকোনো কন্ডিশনে পুরো মেমোরি সিস্টেম লক থাকবে।

লিনিয়ার আর ফুল ডিকোডিংয়ের পার্থক্যটা তাহলে কী হলো?

- লিনিয়ার পদ্ধতিতে (১ম স্লাইড): ৪টি চিপের জন্য প্রসেসরের ৪টি আলাদা লাইন (A_{10} , A_{11} , A_{12} , A_{13}) খরচ হয়ে গিয়েছিল।
- ডিকোডার পদ্ধতিতে (এই স্লাইড): প্রসেসরের মাত্র ৩টি লাইন (A_{10} , A_{11} , A_{12}) ব্যবহার করেই ডিকোডারের মাধ্যমে ৪টি চিপকে কন্ট্রোল করা যাচ্ছে। উপরন্তু, বাকি লাইনগুলোকে এনাবল পিনে আটকে দেওয়ার কারণে কোনো "Shadow মেমোরি" বা ডুপ্লিকেট অ্যাড্রেস তৈরি হওয়ার সুযোগ নেই।

চল, Main Memory Array Design এ PLD ও PROM অংশটা ধাপে ধাপে পরিস্কারভাবে বুঝে নিই।

◇ Programmable Logic Devices (PLDs)

PLD হলো এমন IC যেখানে:

- একটি AND gate array
- একটি OR gate array
- এবং এগুলো programmable

তিন ধরনের PLD আছে:

1. Programmable Read-Only Memory (PROM)
2. Programmable Array Logic (PAL)
3. Programmable Logic Array (PLA)

⚠ এখানে "programming" মানে সফটওয়্যার লেখা না, বরং hardware connection electrically সেট করা।

◇ PROM (Programmable Read Only Memory)

সাধারণ Read-Only Memory (ROM) এ ডাটা স্থায়ীভাবে থাকে।

কিন্তু PROM:

- একবার ব্যবহারকারী নিজে electrically program করতে পারে
- পরে আর পরিবর্তন করা যায় না (One-time programmable)

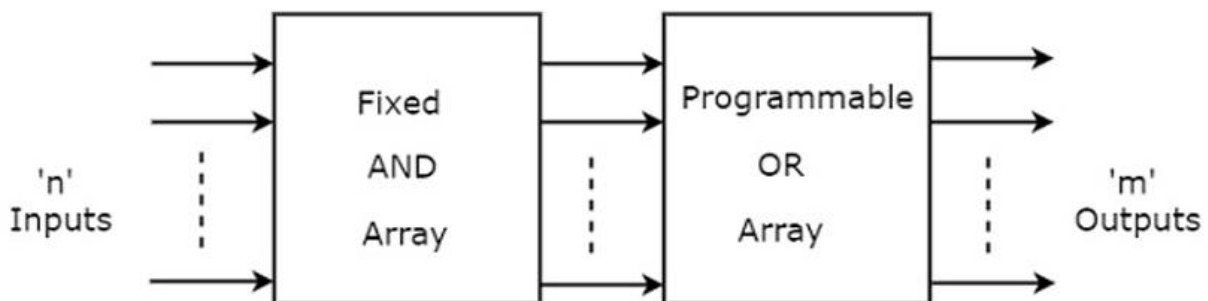
◇ PROM এর গঠন

PROM এ থাকে:

- ☒ Fixed AND array
- ☒ Programmable OR array

মানে:

- AND side programmable না
- OR side programmable



◇ PROM কীভাবে কাজ করে?

ধরা যাক:

Input variable = n টি

তাহলে:

- 2^n টি AND gate লাগবে
- প্রতিটি AND gate $\rightarrow$ একটি minterm তৈরি করবে

এগুলো তৈরি করা হয় সাধারণত একটি **n-to- 2^n decoder** দিয়ে।

👉 Decoder = সব possible minterm তৈরি করে

তারপর:

- সব minterm $\rightarrow$ OR gate এ যায়
- যে minterm দরকার শুধু সেটার connection রাখা হয়
- বাকিগুলো disconnect করা হয়

ফলাফল হবে:

Output = Sum of minterms

Let us implement the following **Boolean functions** using PROM.

$$A(X, Y, Z) = \sum m(5, 6, 7)$$

$$B(X, Y, Z) = \sum m(3, 5, 6, 7)$$

12 Example: 3 Variable Function

ধরা যাক Boolean function এ 3টি variable আছে:

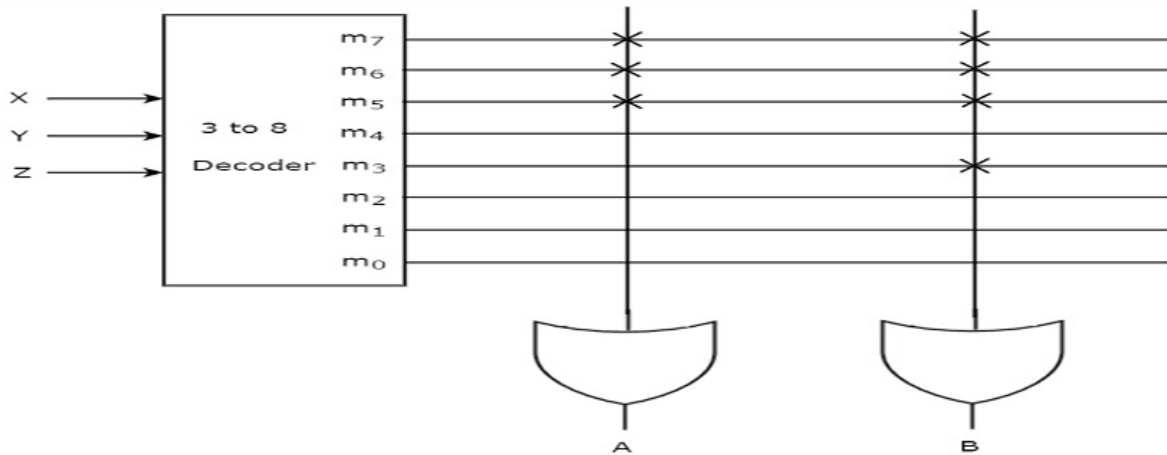
X, Y, Z

তাহলে:

$2^3 = 8$ টি minterm

সুতরাং দরকার:

- একটি 3-to-8 decoder
- 2টি programmable OR gate (কারণ 2টি function implement করবো)



◇ 3-to-8 Decoder কী করে?

Input: X, Y, Z

Output: m0, m1, m2, ..., m7

এগুলো হলো:

$m_0 = X'Y'Z'$

$m_1 = X'Y'Z$

$m_2 = X'YZ'$

...

$m_7 = XYZ$

এই 8টি minterm PROM এর AND array থেকে আসে।

◇ OR Gate Programming

প্রতিটি OR gate:

- সব 8টি minterm access করতে পারে
- কিন্তু শুধু দরকারি minterm গুলো connect করা হয়

Diagram এ “X” চিহ্ন দিয়ে programmable connection দেখানো হয়।

যেমন:

$$F1 = m1 + m3 + m5$$

$$F2 = m0 + m4 + m7$$

তাহলে:

- OR1 এ m1, m3, m5 connect
 - OR2 এ m0, m4, m7 connect
-

◇ কেন PROM ব্যবহার করা হয়?

Main memory array design বা address decoding এ:

- Complex chip select logic তৈরি করতে
 - একাধিক Boolean condition implement করতে
 - কম chip ব্যবহার করে flexible design করতে
-

চল, এবার **Programmable Array Logic (PAL)** সহজভাবে পরিষ্কার করি।

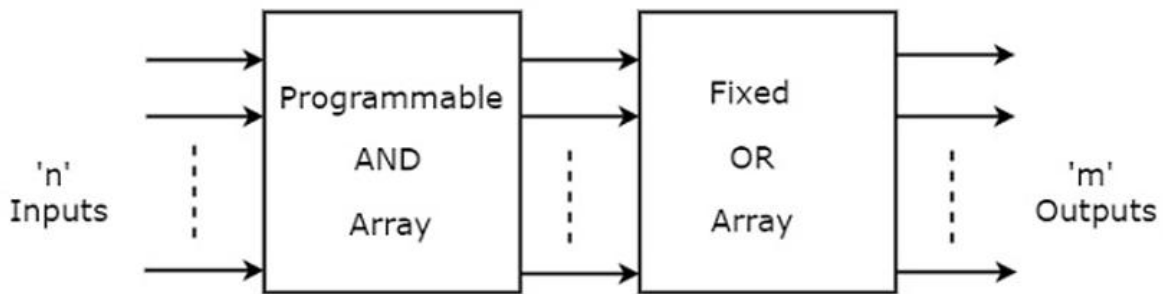
◇ PAL কী?

Programmable Array Logic (PAL) হলো একটি programmable logic device যেখানে:

- AND array → Programmable
- OR array → Fixed

মানে:

- 👉 Product term তৈরি করা যায় flexible ভাবে
- 👉 কিন্তু OR gate এর input সংখ্যা fixed থাকে



◇ PAL এর মূল সুবিধা

PROM এ সব minterm তৈরি করতে হয় (2^n টি)
কিন্তু PAL এ:

- ✓ শুধু প্রয়োজনীয় product term তৈরি করা হয়
- ✓ সব minterm তৈরি করতে হয় না
- ✓ Hardware কম লাগে
- ✓ দ্রুত কাজ করে

◇ PAL এর কাজের ধাপ

ধরা যাক 3টি variable আছে:

X, Y, Z

প্রতিটি AND gate এ থাকবে:

X, X'

Y, Y'

Z, Z'

অর্থাৎ normal এবং complemented input দুটোই available।

এখন প্রয়োজন অনুযায়ী:

- যে literal দরকার সেটি connect করা হবে
- বাকিগুলো disconnect থাকবে

এভাবে একটি AND gate একটি product term তৈরি করবে।

◇ OR Array (Fixed)

PAL এ:

- OR gate এর input সংখ্যা fixed
- Product term গুলো OR gate এ যাবে
- Output হবে Sum of Products (SOP)

| | | |---|---| | 1 | 2 | | 3 | 4 | Example

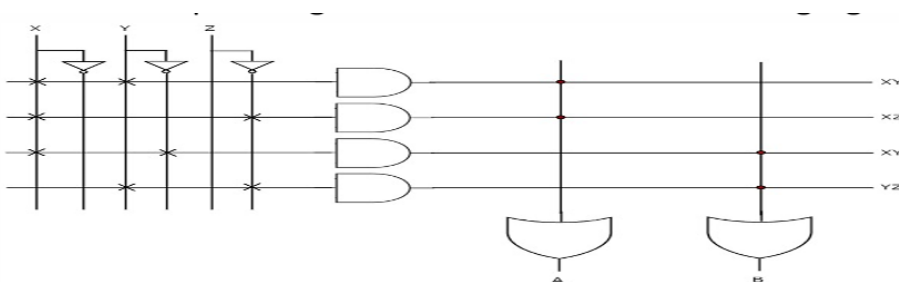
Let us implement the following **Boolean functions** using PAL.

$$A = XY + XZ'$$

$$B = XY' + YZ'$$

তাহলে লাগবে:

- 4টি programmable AND gate
- 2টি fixed OR gate



কীভাবে কাজ করবে?

1. প্রতিটি AND gate → একটি product term তৈরি করবে
2. প্রয়োজনীয় product term → নির্দিষ্ট OR gate এ যাবে
3. OR gate → final Boolean function দিবে

Diagram এ:

- Programmable connection → “X” চিহ্ন
- Fixed connection → “.” চিহ্ন

◇ PROM বনাম PAL পার্থক্য

বিষয়	PROM	PAL
AND Array	Fixed	Programmable
OR Array	Programmable	Fixed
Output Form	Sum of Minterms	Sum of Products
Efficiency	সব minterm তৈরি হয়	শুধু প্রয়োজনীয় product term তৈরি হয়

◇ PAL কেন Memory Design এ গুরুত্বপূর্ণ?

Memory address decoding এ:

- নির্দিষ্ট address combination detect করতে হয়
- Chip select signal তৈরি করতে হয়
- জটিল Boolean logic implement করতে হয়

PAL ব্যবহার করলে:

- ✓ কম gate দিয়ে efficient decoding করা যায়
- ✓ Custom chip select logic বানানো যায়

চল, এবার **Programmable Logic Array (PLA)** পুরোটা পরিষ্কারভাবে বুঝে নিই।

◇ PLA কী?

Programmable Logic Array (PLA) হলো এমন একটি PLD যেখানে:

- AND array → Programmable
- OR array → Programmable

তাই এটি সবচেয়ে flexible PLD।

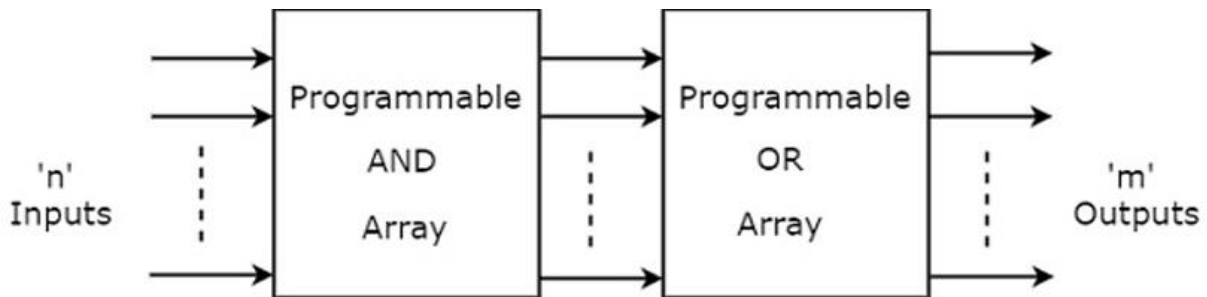
◇ PLA এর গঠন

১ Programmable AND Array

- প্রতিটি AND gate এ থাকে:
 - X, X'
 - Y, Y'
 - Z, Z'

মানে normal এবং complemented দুই ধরনের inputই available।

👉 প্রয়োজন অনুযায়ী literal connect করে product term তৈরি করা যায়।



২ Programmable OR Array

- সব AND gate এর output → সব OR gate এ available
- OR gate এ কোন কোন product term যাবে তা program করা যায়

👉 তাই একাধিক function efficiently implement করা যায়।

◇ Output Form

PLA এর output হয়:

Sum of Products (SOP)

কারণ:

- AND $\rightarrow$ product term
- OR $\rightarrow$ sum

12 34 Example ব্যাখ্যা

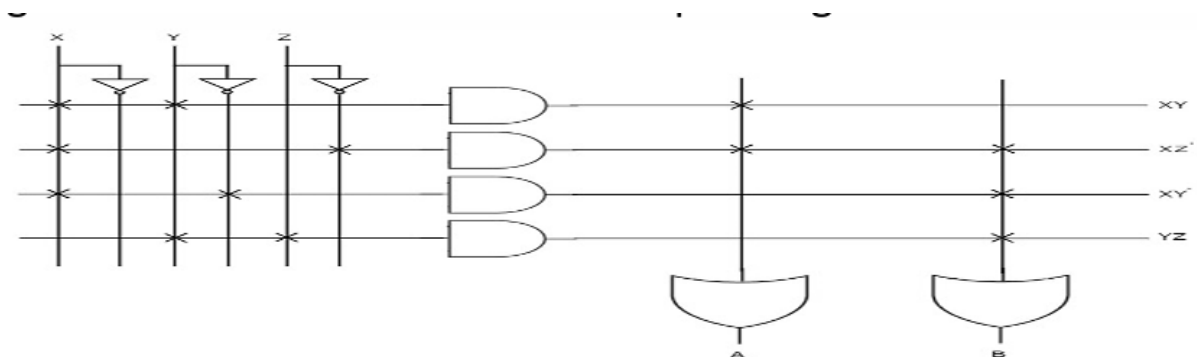
ধরা যাক দুইটি function A ও B দেওয়া আছে।

- A তে 2টি product term
- B তে 3টি product term
- একটি product term দুই function এ common

যেমন:

$$A = XY + XZ'$$

$$B = XY' + YZ + XZ'$$



◇ PLA এর বড় সুবিধা

PROM এ সব minterm তৈরি করতে হয়

PAL এ OR side fixed

কিন্তু PLA এ:

- ✓ শুধু দরকারি product term তৈরি হয়
- ✓ Common product term share করা যায়
- ✓ Hardware efficient
- ✓ Complex logic সহজে implement করা যায়

◇ PROM, PAL, PLA তুলনা

Device	AND Array	OR Array	Flexibility
PROM	Fixed	Programmable	কম
PAL	Programmable	Fixed	মাঝারি
PLA	Programmable	Programmable	সবচেয়ে বেশি

◇ Memory Design এ PLA কেন গুরুত্বপূর্ণ?

Memory address decoding এ:

- Multiple chip select logic
- Complex Boolean condition
- Common address condition reuse

এসব ক্ষেত্রে PLA খুব efficient।

বিশেষ করে যখন:

- কিছু address combination একাধিক memory block এ common থাকে
- তখন PLA hardware কম ব্যবহার করে design সহজ করে।

Combinational PLDs

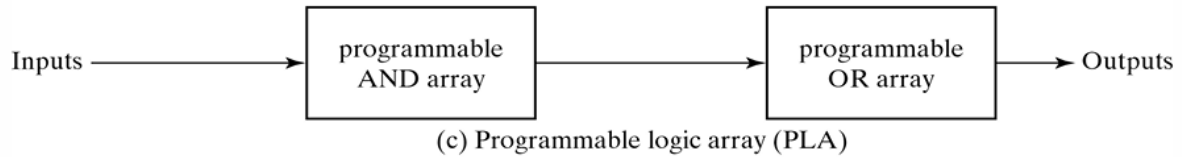
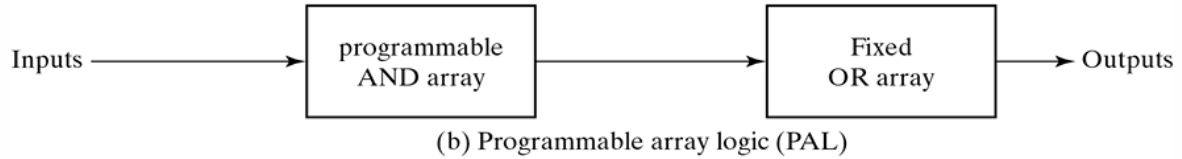
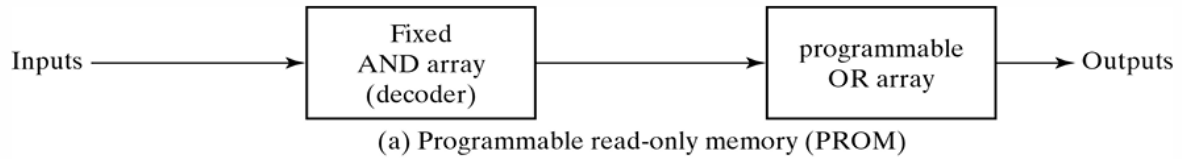


Fig. 7-13 Basic Configuration of Three PLDs